Paradigmas y Lenguajes de Programación III

Trabajo Práctico: “La Web del Terror Accesible”

Desafío Halloween: API REST, auditoría de calidad (ISO/IEC 25010), accesibilidad (WCAG 2.1) y testing de software

Integrantes:
Leandro Acosta
Santiago Romero Baylac
Tomás Romero Baylac

Docente: Dante Sicardi

Fecha de entrega: 10/11/2026

1. Introducción

El trabajo tuvo dos partes. La primera fue el Desafío de Halloween: una aplicación web con API REST donde los usuarios se registran, inician sesión y votan su disfraz favorito, con una página de administración para gestionar los disfraces. La segunda fue convertir esa aplicación en un producto accesible y seguro: cumplir las pautas WCAG 2.1 nivel AA, que se pueda usar solo con el teclado, llegar a 100 puntos en Lighthouse y completar la ficha de calidad según la norma ISO/IEC 25010.

Partimos del código que teníamos en el repositorio del equipo. Al revisarlo con las dos consignas juntas vimos que faltaban partes del desafío original y que había problemas de accesibilidad y seguridad, así que trabajamos sobre todo eso a la vez.

2. Situación inicial

La página principal daba 91 en accesibilidad en Lighthouse y las páginas de login y registro entre 89 y 92. Los problemas que marcaba eran el contraste de los botones (letra blanca sobre naranja, 2,9:1, cuando el mínimo es 4,5:1) y la falta de la etiqueta `<main>` en login y registro.

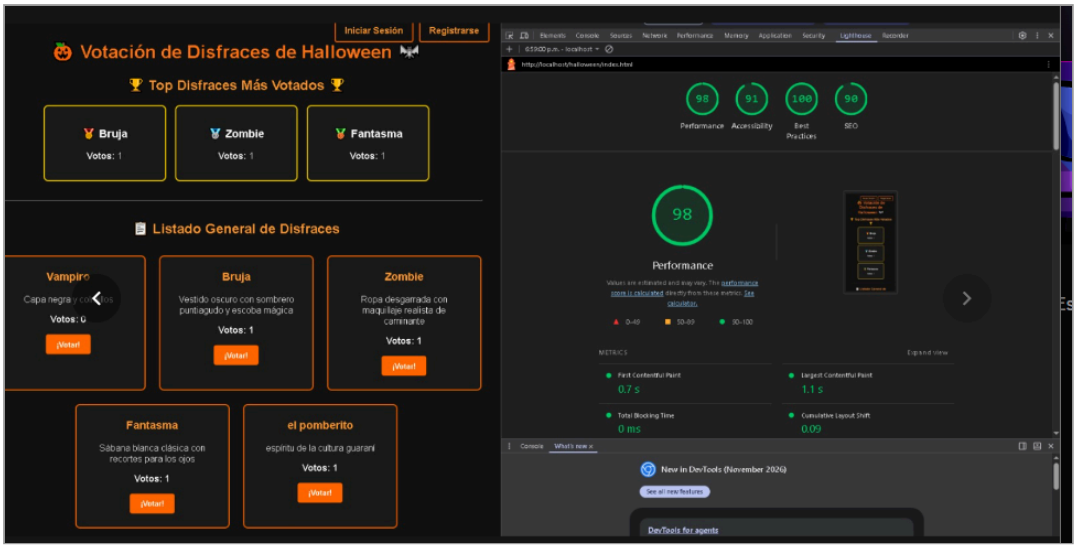


Figura 1. Lighthouse de la versión original (accesibilidad 91).

Comparando el código con la consigna del desafío encontramos además:

- No existía la página de administración ni el CRUD de disfraces.
- No se podían cargar disfraces nuevos ni sus fotos.
- Las tablas no seguían la estructura pedida (por ejemplo, `usuarios` no tenía la columna `clave`).
- Las contraseñas se guardaban en texto plano.
- Había dos logins que no eran compatibles: `login.html` mandaba el nombre y `api/login.php` esperaba un email, así que el login nunca funcionaba.
- Las tarjetas se armaban con `innerHTML` y los datos de la base sin escapar, lo que permite inyectar código (XSS).
- Cuando fallaba una consulta se le mostraba al usuario el error de la base de datos.

- El repositorio no tenía el script para crear la base.

3. Funcionalidades del desafío

3.1 Base de datos

Creamos `database/halloween.sql` con las tres tablas de la consigna y sus columnas. Les agregamos claves primarias con `AUTO_INCREMENT`, claves foráneas en `votos` y dos restricciones `UNIQUE`: una en `usuarios.nombre` para que no haya nombres repetidos y otra en `votos.id_usuario` para que la propia base impida votar dos veces. El único cambio de tipo fue `foto_blob`: usamos `MEDIUMBLOB` en lugar de `BLOB`, porque un `BLOB` admite solo 64 KB y no alcanza para una foto. En `foto` se guarda el tipo de imagen (por ejemplo `image/png`) y en `foto_blob` los bytes. El script también crea el usuario administrador.

3.2 API REST

Método y ruta	Qué hace	Quién	Códigos
GET disfraces.php	Lista de disfraces	Todos	200
GET disfraces.php?id=N	Un disfraz	Todos	200, 404
POST disfraces.php	Crear disfraz con foto	Usuario registrado	201, 400, 401
POST disfraces.php?id=N	Editar disfraz	Administrador	200, 400, 401, 403, 404
DELETE disfraces.php?id=N	Eliminar (borrado lógico con `eliminado = 1`)	Administrador	200, 401, 403, 404
GET foto.php?id=N	Devuelve la imagen guardada en la base	Todos	200, 404
POST registro.php / login.php	Registro e inicio de sesión con nombre de usuario y contraseña	Todos	201 / 200, 400, 401, 409
POST votar.php	Votar	Usuario registrado	200, 400, 401, 404, 409
GET sesion.php / POST logout.php	Estado de la sesión / cerrar sesión	Todos	200

Para editar usamos POST y no PUT porque PHP solo procesa los formularios con archivos (`multipart/form-data`) cuando llegan por POST.

3.3 Páginas

- **index.html:** lista los disfraces con foto, descripción y votos, consumiendo la API. Muestra un podio con los tres más votados y un botón “Votar” por disfraz.
- **registro.html y login.html:** nombre de usuario y contraseña, como pide la consigna.
- **subir.html:** un usuario registrado carga su disfraz con nombre, descripción y foto.

- **admin.html:** solo para el usuario `admin`. Tiene un formulario para agregar o editar y una tabla con todos los disfraces y los botones Editar y Eliminar. Si entra otro usuario, ve un aviso de que la página es solo para el administrador; además la API rechaza con 403 cualquier edición que no venga del administrador, aunque se llame directo a la URL.

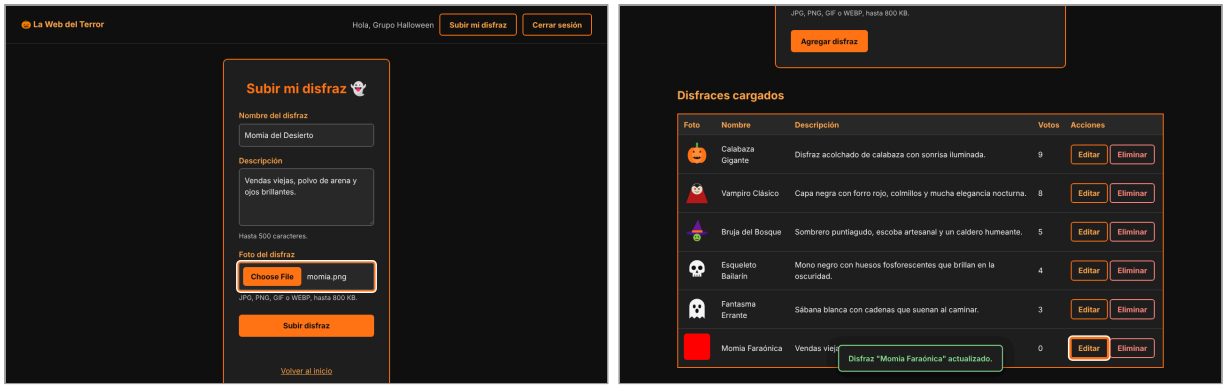


Figura 2. Página para subir un disfraz y panel de administración después de editar uno.

4. Cambios de accesibilidad

4.1 Correcciones de Lighthouse

Pasamos todos los colores a variables en `css/estilos.css`. En los botones dejamos el mismo naranja con letra oscura (`#121212`) y el contraste subió a 7:1. Revisamos el resto de los colores con la fórmula de WCAG:

Elemento	Colores	Contraste	Mínimo
Botón original	blanco sobre #ff6600	2,94:1	4,5:1 (no cumplía)
Botón corregido	#121212 sobre #ff7518	6,97:1	4,5:1
Título principal	#ff7518 sobre #121212	6,97:1	4,5:1
Nombres de los disfraces	#ffa94d sobre #1e1e1e	8,76:1	4,5:1
Descripciones	#c8c8c8 sobre #1e1e1e	9,96:1	4,5:1
Botón Eliminar y errores	#ff8a80 sobre #1e1e1e	7,30:1	4,5:1
Borde de los campos	#8a8a8a sobre #2a2a2a	4,16:1	3:1

Todas las páginas tienen `<header>`, `<nav>`, `<main>` y un solo `<h1>`. Todos los campos tienen `<label for="...">`, incluido el de la foto, y la ayuda de cada campo se asocia con `aria-describedby`. Todas las fotos tienen un `alt` con el nombre del disfraz, y los emojis decorativos llevan `aria-hidden="true"`. La tabla de administración tiene `<caption>` y encabezados con `scope="col"`.

4.2 Navegación con teclado

Sacamos los `onclick` del HTML y usamos solamente botones y enlaces nativos. Agregamos un enlace “Saltar al contenido principal” que aparece con el primer Tab y un borde blanco con sombra naranja usando `:focus-visible`. Los botones repetidos tienen un texto oculto con el

nombre del disfraz, así el lector dice “Votar por Bruja del Bosque” o “Editar Bruja del Bosque” y no solo “Votar” o “Editar”.

También cuidamos a dónde va el foco cuando la página cambia: después de votar vuelve al mismo botón; al editar en la administración pasa al campo del nombre y al guardar vuelve al botón “Editar” de esa fila; al eliminar pasa al título de la tabla; si un formulario tiene un error, el foco va al campo con el error.

Probamos todo el recorrido usando solamente el teclado: registrarse, iniciar sesión, votar, subir un disfraz, cerrar sesión y, como administrador, editar y eliminar un disfraz. El campo de la foto también se abre con Enter o Espacio. Este es el orden de tabulación de la portada:

Tab	Elemento	Foco visible
1	Saltar al contenido principal	Sí
2	Logo “La Web del Terror”	Sí
3 y 4	Iniciar sesión, Registrarse (con sesión: Subir mi disfraz, Administración, Cerrar sesión)	Sí
5 en adelante	Votar por cada disfraz, en el orden de la lista	Sí

Además analizamos con la herramienta axe nueve estados distintos (portada con y sin sesión, errores de formulario, subir disfraz, administración antes y después de editar) y no marcó ningún error de WCAG 2.1 AA.

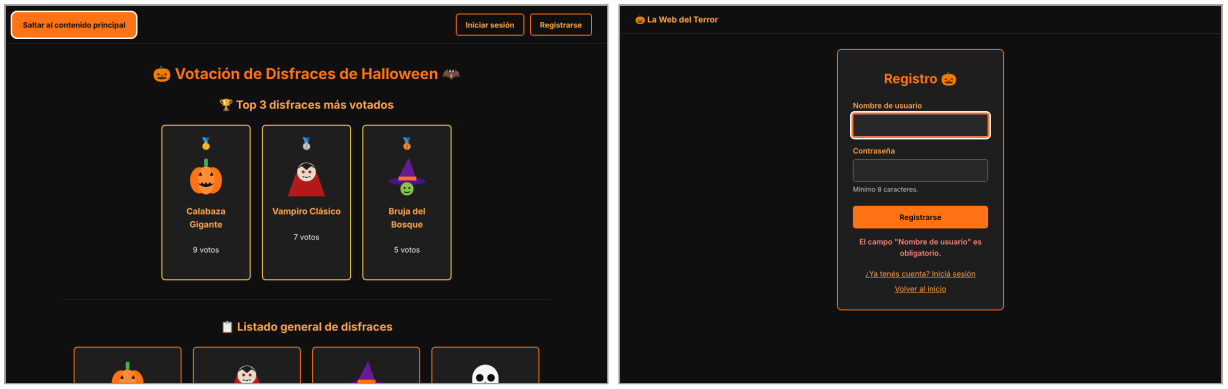


Figura 3. Enlace para saltar al contenido y error de validación en el registro, con el foco en el campo vacío.

4.3 Mensajes de las peticiones fetch()

Reemplazamos los `alert()` por contenedores con `aria-live="polite"`. Las respuestas de la API se escriben ahí con `.textContent`, así el lector de pantalla las lee sin que el usuario pierda el lugar donde estaba:

```
export function anunciar(region, mensaje, tipo = 'ok') {
    region.classList.remove('ok', 'error');
    region.textContent = '';
    setTimeout(() => {
        region.classList.add(tipo);
        region.textContent = mensaje;
    }, 50);
}

<div id="aviso" class="aviso" role="status" aria-live="polite"></div>
```

Usar `.textContent` en vez de `innerHTML` también cerró el problema de XSS, porque el texto nunca se interpreta como HTML.

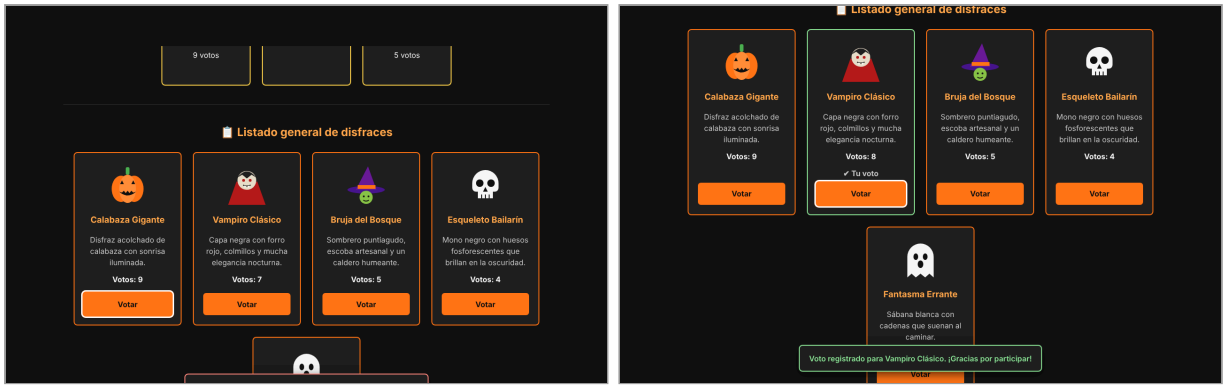


Figura 4. Mensaje al votar sin sesión y mensaje de voto registrado.

4.4 Resultado de Lighthouse

Página	Accesibilidad antes	Accesibilidad después	Rendimiento	Buenas prácticas	SEO
index.html	91	100	100	100	100
login.html	92	100	100	100	100
registro.html	92	100	100	100	100
subir.html	(no existía)	100	100	100	100
admin.html (como administrador)	(no existía)	100	99	100	100

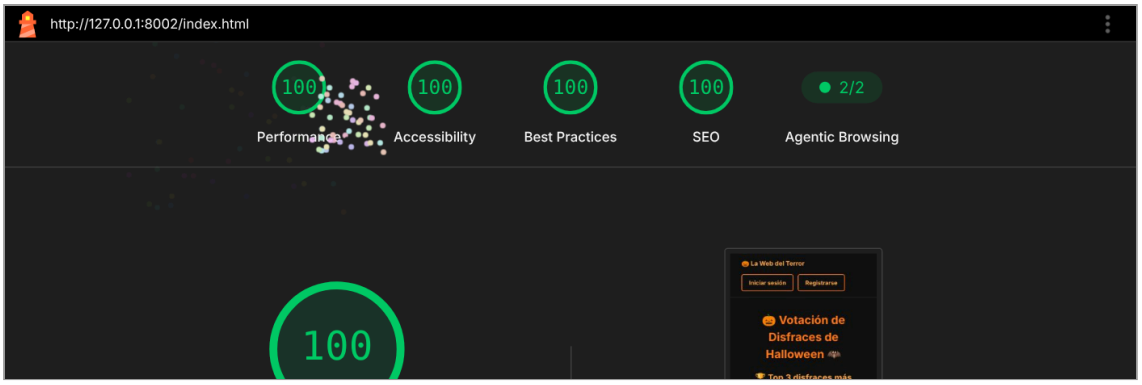


Figura 5. Lighthouse de index.html.

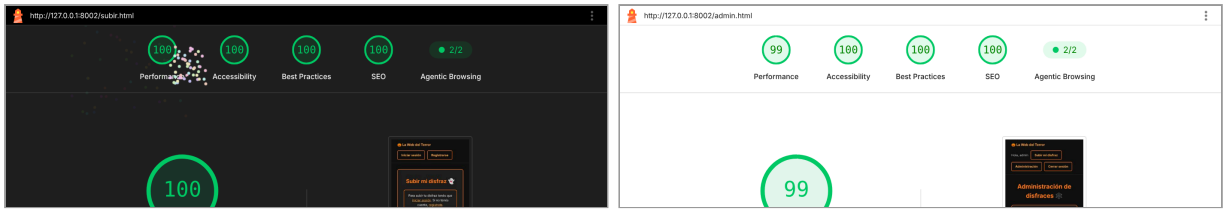


Figura 6. Lighthouse de subir.html y de admin.html con sesión de administrador.

Si se corre Lighthouse desde Chrome con extensiones instaladas, el rendimiento puede bajar porque las extensiones se cargan junto con la página. Por eso lo corremos en una ventana de incógnito.

5. Seguridad

- **Contraseñas:** se guardan con `password_hash($clave, PASSWORD_BCRYPT)` en la columna `clave` y se verifican con `password_verify()`.
- **Inyección SQL:** todas las consultas usan sentencias preparadas con `prepare()` y `execute()`.
- **XSS:** el DOM se arma con `.textContent`; no hay ningún `innerHTML` en el proyecto.
- **Sesión:** se regenera el id al iniciar sesión y la cookie es `HttpOnly` y `SameSite=Lax`.
- **Permisos:** `exigirLogin()` y `exigirAdmin()` en `api/lib/auth.php`. Votar y subir piden sesión (401); editar y eliminar piden ser administrador (403).
- **Fotos:** no confiamos en la extensión del archivo. `getimagesize()` lee el contenido y solo acepta JPG, PNG, GIF o WEBP de hasta 800 KB. Al mostrar la imagen, `foto.php` envía una política de seguridad que impide ejecutar código aunque se colara un archivo raro.
- **Errores:** el detalle técnico va a `error_log()` y el usuario solo ve un mensaje general. El login responde siempre “Usuario o contraseña incorrectos”, sin decir cuál de los dos falló.

6. Pruebas de la API

Hicimos un script en PHP (`tests/test_api.php`) que llama a cada endpoint y revisa que devuelva el código HTTP esperado. Se corre con `C:\xampp\php\php.exe tests\test_api.php http://localhost/halloween`.

N°	Prueba	Esperado	Obtenido	Resultado
1	GET disfraces devuelve lista JSON	200	200	OK

2	GET un disfraz por id	200	200	OK
3	GET disfraz inexistente	404	404	OK
4	Foto guardada en la base (foto_blob)	200	200	OK
5	PUT no permitido en disfraces	405	405	OK
6	Registro con JSON inválido	400	400	OK
7	Registro con campos vacíos	400	400	OK
8	Registro con contraseña corta	400	400	OK
9	Registro correcto	201	201	OK
10	Registro con nombre repetido -> conflicto	409	409	OK
11	Votar sin sesión -> no autorizado	401	401	OK
12	Subir disfraz sin sesión -> no autorizado	401	401	OK
13	Eliminar sin sesión -> no autorizado	401	401	OK
14	Login con contraseña incorrecta	401	401	OK
15	Intento de inyección SQL en login	401	401	OK
16	Login correcto (usuario común)	200	200	OK
17	Votar con id inválido	400	400	OK
18	Votar disfraz inexistente	404	404	OK
19	Voto válido	200	200	OK
20	Segundo voto del mismo usuario -> conflicto	409	409	OK
21	Subir archivo que no es imagen	400	400	OK
22	Subir disfraz sin foto	400	400	OK
23	Usuario sube su disfraz con foto (Create)	201	201	OK
24	Editar siendo usuario común -> prohibido	403	403	OK
25	Eliminar siendo usuario común -> prohibido	403	403	OK
26	Cerrar sesión	200	200	OK
27	Login del administrador	200	200	OK
28	Admin edita nombre y descripción (Update)	200	200	OK
29	El cambio se guardó	200	200	OK
30	Admin edita con nombre vacío	400	400	OK
31	Admin elimina el disfraz (Delete)	200	200	OK
32	El disfraz eliminado ya no aparece	404	404	OK
33	Cerrar sesión del administrador	200	200	OK

Pasaron las 33 pruebas. El tiempo promedio de respuesta fue de 9,1 ms y el máximo de 71,3 ms. Los pedidos más lentos son el registro y el login, porque BCRYPT tarda a propósito para que sea difícil adivinar contraseñas por fuerza bruta.

7. Ficha de autoevaluación ISO/IEC 25010

Característica	Evidencia en el código	Estado
1. Adecuación funcional	El backend en PHP guarda usuarios, votos y disfraces en MySQL con PDO (`api/config/conexion.php`). Funcionan el registro y el login con nombre de usuario, la votación con un voto por usuario, la carga de disfraces con foto y el CRUD del administrador.	Cumple
2. Eficiencia de desempeño	La API responde en 9,1 ms en promedio según las pruebas. Lighthouse da 100 en rendimiento en las páginas públicas.	Cumple
3. Compatibilidad	El frontend es HTML y JavaScript sin frameworks y se comunica con el backend solo por `fetch()`. La API devuelve JSON.	Cumple
4. Fiabilidad	Se capturan las excepciones PDO y se devuelven los códigos 200, 201, 400, 401, 403, 404, 405, 409 y 500. El voto usa transacción con `rollBack()`.	Cumple
5. Seguridad (OWASP)	Contraseñas con BCRYPT, sentencias preparadas (probado con `' OR '1'='1`), `.textContent` contra XSS, sesión regenerada, fotos validadas por contenido y permisos de administrador.	Cumple
6. Usabilidad / accesibilidad	Lighthouse da 100/100 en accesibilidad en las cinco páginas y todo el sitio se puede usar solo con Tab y Enter, incluida la administración.	Cumple
7. Mantenibilidad	Código separado en capas (conexión, utilidades y permisos, endpoints, un script por página y una sola hoja de estilos), con pruebas y versionado en Git.	Cumple

8. Conclusión

La característica que más trabajo nos llevó fue la seguridad. Al principio pensábamos que alcanzaba con arreglar los colores para llegar a 100 en Lighthouse, pero al completar la ficha vimos que no podíamos poner “Cumple” si las contraseñas estaban sin cifrar y el login ni siquiera funcionaba. Rehicimos el registro y el login con BCRYPT y nombre de usuario, como pedía el desafío. Después, al agregar la carga de fotos, aparecieron riesgos nuevos: tuvimos que validar las imágenes por su contenido y separar bien qué puede hacer un usuario común y qué solo el administrador.

También aprendimos que Lighthouse no revisa todo. Los problemas de foco al recargar la página, los botones con el mismo nombre repetido o hacia dónde va el foco después de eliminar una fila solo los encontramos usando el sitio con el teclado.

9. Instrucciones para ejecutar

1. Copiar la carpeta `halloween` en `C:\xampp\htdocs` y arrancar Apache y MySQL.

2. En phpMyAdmin, ir a Importar y elegir `halloween/database/halloween.sql`.
3. Entrar a `http://localhost/halloween/index.html`. Usuario administrador: `admin`, contraseña `Admin2026!`.

Repositorio: https://github.com/tomasromerobaylac/Proyecto-Halloween